Tiny Changes, Big Trouble:

Adversarial Attacks and Defenses in Deep Learning Models

Zhipeng Cai, Liran Ma

Copyright © 2026 Zhipeng Cai, Georgia State University; Liran Ma, Miami University

The development of this document was supported in part by the National Science Foundation's SaTC EDU program. Any opinions, findings, and conclusions or recommendations expressed in this document are those of the author(s) and do not necessarily reflect the views of the National Science Foundation.

Learning Goals

By the end of this lab, you should be able to:

- Explain why a small, invisible change to an image can completely fool a neural network.
- Describe the FGSM attack in plain language, without referring to code.
- Explain why PGD is a stronger attack than FGSM.
- Read an accuracy-vs-epsilon curve and say what it means.
- Compare two defenses and describe the tradeoff each one makes.

Contents

1 Introduction **3**

2 Setting Up: What We Are Working With **3**
2.1 Why MNIST? 4
2.2 What "solving" MNIST looks like 4

3 What the Model Actually Sees: Images Are Numbers **7**
3.1 The pixel grid 7
3.2 Gradients: the model's sensitivity map 7

4 Attack 1: FGSM — One Step Is Enough **8**
4.1 The idea behind FGSM 8
4.2 Why the sign() matters: a small numerical example 8
4.3 Running FGSM 9
4.4 Visualizing the attack: what the perturbation looks like 13

5 Why Does FGSM Work So Well? **14**

6 Attack 2: PGD — Many Steps, Stronger Attack **15**
6.1 The limitation FGSM ignores 15

7 Defense 1: Adversarial Training **19**
7.1 The simplest idea: show the model what it will face 19

8 Defense 2: Feature Squeezing **22**
8.1 A completely different philosophy 22

9 Putting It All Together **25**
9.1 The defense comparison table 25
9.2 A practical decision guide 26

10 Summary **27**

1 Introduction

It is 11:47 PM in the radiology wing of Meridian General Hospital. An AI triage system, which is trained on 200,000 chest X-rays, scans each incoming image and flags critical cases for immediate review.

A chest X-ray arrives. Every radiologist in the room would recognize it as pneumonia. The image quality is fine. The patient's history is documented. But the AI returns a confident verdict:

97% healthy. No action needed.

Three hours later, the patient is in intensive care.

Investigators later discover that the image had been tampered with, but not in any way a human could detect. Thousands of individual pixels had been shifted by amounts smaller than the noise grain of the original scan. To the human eye, nothing changed. To the AI, everything changed.

Those shifts were not random. They were *calculated*.

This is an **adversarial attack**: a tiny, mathematically precise perturbation to an input that causes a neural network to fail—confidently and silently.

This lab explains how these attacks work, why they are so effective, and what can be done to resist them. We start with the simplest possible model and dataset (handwritten digit recognition on MNIST) so every piece of the mechanism is visible.

Think before you read on: What do you hypothesize?

The AI model was trained on 200,000 images and tested at 98% accuracy. It was not hacked after deployment. The pixel changes are invisible to humans. Yet it failed confidently. Write your hypothesis below before reading further. There are no wrong answers at this stage.

What might cause a well-trained model to be confidently wrong on an image that looks completely normal to humans?

_____

_____

You will return to this hypothesis at the end of the lab.

2 Setting Up: What We Are Working With

2.1 Why MNIST?

The hospital story uses chest X-rays. This lab uses **MNIST**—a dataset of handwritten digits 0–9, each image 28×28 pixels in grayscale.

We use MNIST for two reasons. First, the images are tiny (784 pixels each), so every computation runs quickly on a laptop CPU. Second, and more importantly, the small size makes the mathematics and the visualizations fully transparent. The same core ideas extend to medical images, face recognition, and autonomous-driving perception, although the data scale, perceptual constraints, and threat models differ substantially across domains.

2.2 What "solving" MNIST looks like

We train a small convolutional neural network (CNN). It learns to look at the pixel grid and decide which digit (0–9) is most likely. In practice, only 3 epochs are enough to reach 98% on MNIST. In this lab, however, we train for longer so that you can clearly observe the full training-loss trend and see how the optimization gradually stabilizes.

That accuracy is our **baseline**—the starting point before any attack.

A word about the code

Each code block in this lab is a self-contained **cell** meant to be run in order in Google Colab. You do not need to understand every line before running it. Read the comment above each section, run the cell, observe the output, and then answer the questions that follow. The understanding comes from the combination of running and reflecting.

Listing 1: Cell 1: Imports and device setup

```
import torch
import torch.nn as nn
...

# Global figure style
mpl.rcParams.update({
    'font.family':        'serif',
 ...
    'axes.spines.right':  True,
})
```

Listing 2: Cell 2: Load MNIST and define the CNN

```
transform    = transforms.ToTensor()
trainset     = torchvision.datasets.MNIST(root='./data', train=True,
                                          download=True, transform=transform)
testset      = torchvision.datasets.MNIST(root='./data', train=False,
                                          download=True, transform=transform)

class SimpleCNN(nn.Module):
    """
    Two convolutional layers followed by two fully-connected layers.
    Input:  1 x 28 x 28 grayscale image
    Output: 10 class scores (one per digit)
    """
criterion = nn.CrossEntropyLoss()
```

Listing 3: Cell 3: Train the model (about 2 minutes on CPU)

```
model     = SimpleCNN().to(device)
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)

epoch_losses = []   # store average loss of each epoch
num_epochs = 20  # 3 epochs are already enough for strong MNIST accuracy; we use 20
    here only to visualize a clearer loss curve.
for epoch in range(num_epochs):
    model.train()
    running_loss = 0.0
    for imgs, lbls in train_loader:
        imgs, lbls = imgs.to(device), lbls.to(device)
        optimizer.zero_grad()
        loss = criterion(model(imgs), lbls)
        loss.backward()
        optimizer.step()
        running_loss += loss.item()
    avg = running_loss / len(train_loader)
    epoch_losses.append(avg)
    print(f'Epoch {epoch+1}/{num_epochs}  |  avg loss: {avg:.4f}')

# Evaluate on the test set
model.eval()
```

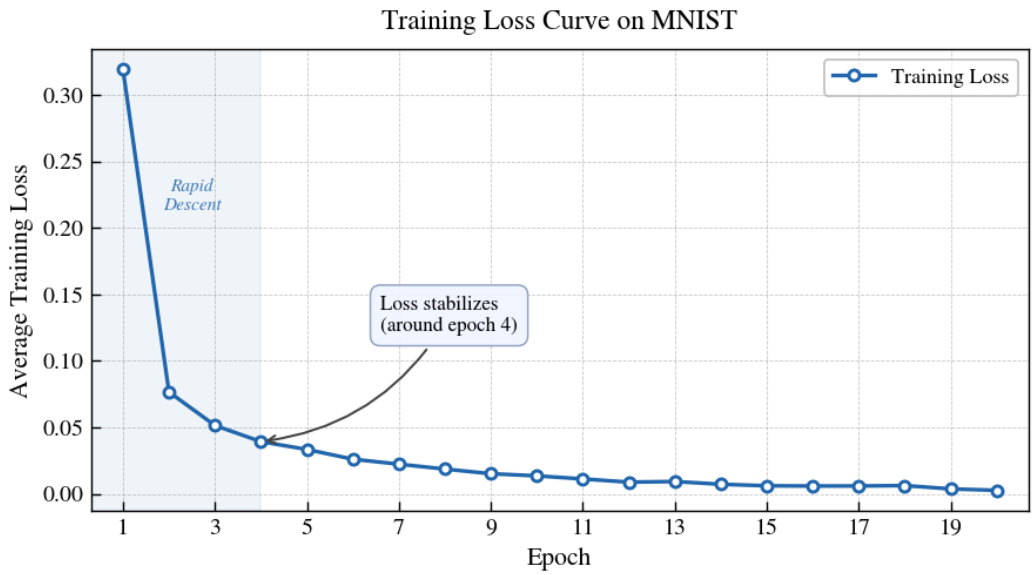
What you should see

With the settings above, the training loss should decrease steadily and the clean test accuracy should reach about 98%–99%. In practice, only 3 epochs are enough for strong MNIST performance. Here we train longer mainly so that we can see a clearer loss curve and observe how the optimization gradually stabilizes.

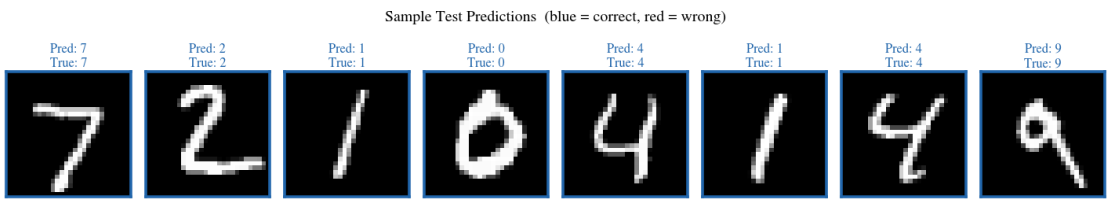
Before moving to adversarial attacks, we first check that the model behaves normally on clean data. The example output below combines a few representative training logs with two visual sanity checks: a training-loss curve and a small set of clean test predictions. The loss curve drops sharply in the early epochs and then decreases more gradually, which is the typical pattern of stable learning on MNIST. The sample predictions provide a quick sanity check that the trained CNN already classifies ordinary handwritten digits correctly on clean test inputs.

Example Output

```
Epoch 1/20   |  avg loss:   0.3191
Epoch 3/20   |  avg loss:   0.0514
Epoch 10/20  |  avg loss:   0.0135
Epoch 20/20  |  avg loss:   0.0026
   Clean test accuracy:  99.13%
```



Training loss on clean MNIST.



Sample clean test predictions.

This combined result panel answers two basic questions before any attack begins: *Is the model learning stably?* and *Does it already work on normal inputs?* If both answers are yes, then the later adversarial failures become much easier to interpret.

If your accuracy is below 95%, check that the random seed was not changed and re-run from Cell 1.

Exercise 1: Your Baseline

Record your results before any attack.

Clean test accuracy: _____ %
Training loss after the final epochs: _____

1. Training loss decreased each epoch. What does a decreasing training loss mean in plain language?
2. The model is tested on images it has *never seen during training*. Why is this the correct way to evaluate it, rather than testing on the training images themselves?
3. Even a small CNN reaches high accuracy on MNIST after only a few epochs. In this lab, we train a bit longer mainly to visualize the loss curve more clearly. Why might a larger,

better-trained model actually be *easier* to attack in some ways? (Hint: think about what a model learns more precisely as it improves.)

3 What the Model Actually Sees: Images Are Numbers

3.1 The pixel grid

A 28×28 grayscale MNIST image is just a grid of 784 numbers. Each number is a pixel intensity between 0 (black) and 1 (white). The model receives these 784 numbers, passes them through layers of learned arithmetic, and produces 10 output scores—one per digit class. The class with the highest score is the prediction.

This means the model is, fundamentally, a mathematical function:

$$f(\mathbf{x}) = \hat{y}$$

where $\mathbf{x} \in [0,1]^{784}$ is the pixel vector and $\hat{y} \in \{0, 1, \ldots, 9\}$ is the predicted class.

3.2 Gradients: the model's sensitivity map

You trained the model using **gradients**. During training, gradients flow backward from the loss to the model's *weights*, and the weights are nudged to reduce the loss. This is gradient descent.

But here is the key question: what if instead of asking "how should I change the *weights* to reduce the loss," you asked "how should I change the *input pixels* to *increase* the loss?"

The math is identical. You compute the gradient of the loss with respect to the input pixels:

$$\nabla_{\mathbf{x}} \mathcal{L}(f(\mathbf{x}), y)$$

This gradient is a vector of 784 numbers—one for each pixel. Each number tells you: "if this pixel increases by a tiny amount, the loss changes by this much." It is a sensitivity map of the model.

Why does the gradient point toward higher loss?

During training, we move the weights *against* the gradient to *decrease* the loss. An adversarial attack moves the *input along* the gradient to *increase* the loss. Same gradient. Opposite direction. Opposite goal.
This is the core insight: the machinery that makes learning possible is also the machinery that makes the model vulnerable.

A numerical intuition: Scene A

Suppose the gradient for a particular image has these values on two pixels:

- Pixel (14,14): gradient $= +8.3$ (large positive value)
- Pixel (3,7): gradient $= +0.02$ (near-zero value)

Think before continuing: What does each gradient value mean?

1. Pixel (14,14) has gradient $+8.3$. If you increase that pixel by 0.01, by how much does the loss change? In which direction?
2. Pixel (3,7) has gradient $+0.02$. Is this pixel important for the model's decision?
3. If you wanted to hurt the model as much as possible by changing only one pixel by $\varepsilon = 0.1$, which pixel would you choose? In which direction would you push it?

Your answers:

1. _____

3. Pixel _____, direction: _____

4 Attack 1: FGSM — One Step Is Enough

4.1 The idea behind FGSM

The Fast Gradient Sign Method (Goodfellow et al., 2014) extends the intuition above to all 784 pixels at once. Instead of picking one pixel, it nudges *every* pixel simultaneously, each one in the direction that increases the loss most. The step size is exactly ε, the perturbation budget.

The use of $\text{sign}(\cdot)$ is a deliberate choice: it discards the *magnitude* of each gradient and keeps only the *direction* ($+1$ or -1). This ensures every single pixel moves by exactly ε—no more, no less—so the total perturbation is as spread out and as invisible as possible.

The FGSM formula:

$$\mathbf{x}_{\text{adv}} = \mathbf{x} + \varepsilon \cdot \text{sign}\big(\nabla_{\mathbf{x}} \mathcal{L}(f(\mathbf{x}), y)\big)$$

$\mathbf{x}$	the original image (784 pixel values)
ε	the perturbation budget (how far any pixel can move)
$\nabla_{\mathbf{x}}\mathcal{L}$	the gradient of the loss w.r.t. every pixel
$\text{sign}(\cdot)$	$+1$ if the gradient is positive; -1 if negative
$\mathbf{x}_{\text{adv}}$	the resulting adversarial image

Result: every pixel moves in the direction that hurts the model, by exactly ε. The clamp at the end keeps pixel values in $[0, 1]$.

4.2 Why the sign() matters: a small numerical example

Scene A: raw gradient (no sign)

Suppose two pixels have gradients $+8.3$ and $+0.2$. If we multiply the raw gradient by $\varepsilon = 0.1$:

- Pixel with gradient $+8.3$: moves by $0.1 \times 8.3 = 0.83$. (Much larger than ε!)
- Pixel with gradient $+0.2$: moves by $0.1 \times 0.2 = 0.02$. (Tiny.)

Most pixels barely move. A few pixels move dramatically. The perturbation is *concentrated* and more likely to be visible.

Scene B: with sign()

Now apply $\text{sign}(\cdot)$ first. Both gradients become $+1$. Multiply by $\varepsilon = 0.1$:

- Pixel with gradient $+8.3$: moves by exactly 0.1.
- Pixel with gradient $+0.2$: moves by exactly 0.1.

Every pixel contributes equally. The perturbation is *spread out* across all 784 pixels, each invisible individually.

Exercise 2: Scene C — Fill in the blanks

A third image has gradients on three pixels: $+5.1$, -3.4, and $+0.05$.

Complete the table for $\varepsilon = 0.15$:

Pixel	Gradient	sign(gradient)	Pixel change ($\varepsilon \cdot$ sign)
Pixel A	$+5.1$	_____	_____
Pixel B	-3.4	_____	_____
Pixel C	$+0.05$	_____	_____

1. Pixel A has a much larger gradient magnitude than Pixel C. After applying $\text{sign}(\cdot)$, are they treated the same or differently? Is this a feature or a limitation?
2. Pixel B has a *negative* gradient. What does it mean for a pixel's gradient to be negative? What does FGSM do to that pixel?
3. If $\varepsilon = 0$, what is $\mathbf{x}_{\text{adv}}$? What accuracy would you expect?

4.3 Running FGSM

Listing 4: Cell 4: FGSM attack function

```python
def fgsm_attack(image, epsilon, grad):
    """
    Apply one FGSM step.
    Args:
        image   : original input tensor (N, C, H, W)
        epsilon : perturbation budget -- how far any pixel can move
        grad    : gradient of the loss w.r.t. the image pixels
    Returns:
        adversarial image, pixel values clamped to [0, 1]
    """
    sign_grad = grad.sign()            # +1 or -1 for each pixel
    x_adv     = image + epsilon * sign_grad
    return torch.clamp(x_adv, 0, 1)  # keep pixel values valid
```

Listing 5: Cell 5: Evaluate accuracy under FGSM for one epsilon value

```python
def evaluate_fgsm(model, loader, epsilon, n_batches=4):
    """
    Run FGSM on up to n_batches and return:
      - accuracy on adversarial examples
      - a list of example dicts for images where the prediction changed
    """
```

```python

    for batch_idx, (images, labels) in enumerate(loader):
        if batch_idx >= n_batches:
            break
        images, labels = images.to(device), labels.to(device)

        # Step 1: enable gradient tracking on the INPUT (not the weights)
        images.requires_grad_(True)

        # Step 2: forward pass -- compute model output and loss
        outputs = model(images)
        loss    = criterion(outputs, labels)

        # Step 3: backward pass -- compute gradient of loss w.r.t. pixels
        model.zero_grad()
        loss.backward()
        # images.grad now contains the gradient for each pixel

        # Step 4: apply FGSM using the pixel gradient
        adv_images = fgsm_attack(images, epsilon, images.grad.detach())

        # Step 5: re-evaluate the model on the adversarial images
        with torch.no_grad():
            adv_outputs = model(adv_images)
        ...
```

Listing 6: Cell 5b: Evaluate FGSM on a single batch for direct comparison

```python
def evaluate_fgsm_on_batch(model, images, labels, epsilon):
    """
    Run FGSM on a single batch and return:
      - adversarial accuracy on that batch
      - adversarial images
      - original predictions
      - adversarial predictions
    """
    model.eval()

    images = images.clone().detach().to(device)
    ...
```

What the five steps above are doing

Before running the epsilon sweep, make sure you understand what happens in Cell 5:

1. **requires_grad = True**: tells PyTorch to track how the loss changes with respect to the *input*, not just the weights.
2. **forward pass**: compute what the model predicts and how wrong it is (the loss).
3. **backward pass**: compute the gradient of that loss with respect to every pixel.
4. **fgsm_attack**: use those pixel gradients to perturb the image.
5. **re-evaluate**: check whether the perturbed image fools the model.

This function is useful for sweeping over multiple ε values and measuring the overall drop in accuracy. Later, when we compare FGSM and PGD directly, we will also use a smaller single-batch version so that all attack conditions are evaluated on exactly the same inputs.

Listing 7: Cell 6: Sweep epsilon from 0 to 0.30 and plot the accuracy curve

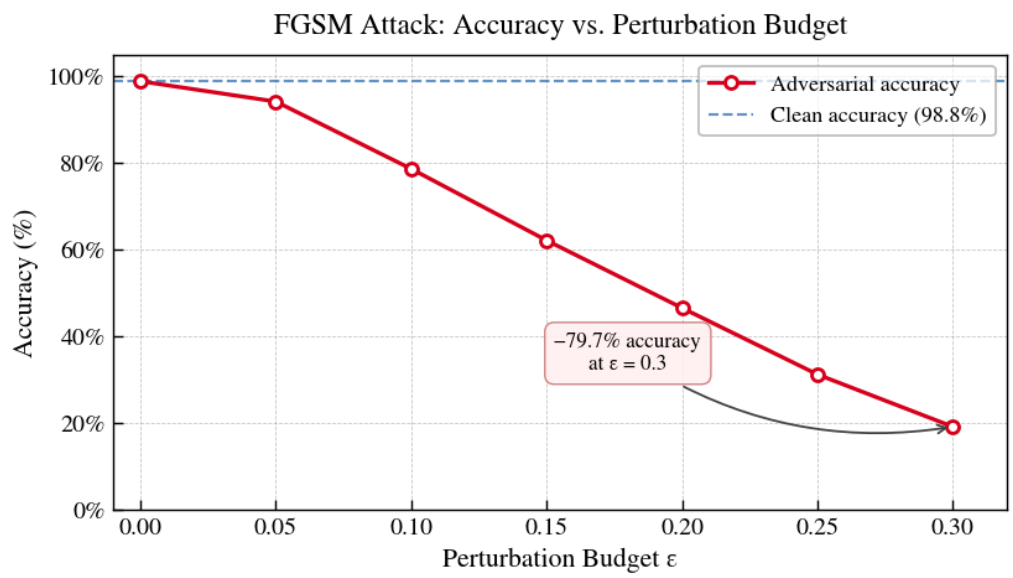
```
# Visualization 3: FGSM accuracy analysis
epsilons = [0.0, 0.05, 0.10, 0.15, 0.20, 0.25, 0.30]
accs     = []
all_exs  = {}   # epsilon -> list of example dicts
for eps in epsilons:
    acc, exs = evaluate_fgsm(model, test_loader, eps, n_batches=8)
    accs.append(acc * 100)
    all_exs[eps] = exs
    ...

# Visualization 3: Accuracy vs epsilon line plot
fig, ax = plt.subplots(figsize=(6.5, 3.8))

# Clean-accuracy reference line (epsilon = 0)
ax.axhline(
    y=accs[0], color='#2166AC', linewidth=1.2,
    linestyle='--', alpha=0.7, label=f'Clean accuracy ({accs[0]:.1f}%)',
)
...
```

What the output should look like

With the settings above, the model's accuracy should decrease steadily as the perturbation budget increases. In other words, stronger FGSM perturbations make it easier to fool the classifier. The example output below combines representative accuracy logs with two visual checks: an accuracy-versus-epsilon curve and a gallery of adversarial examples.

Example Output

```
epsilon = 0.00  ->  adv accuracy:   98.8%
epsilon = 0.05  ->  adv accuracy:   94.1%
epsilon = 0.10  ->  adv accuracy:   78.6%
epsilon = 0.15  ->  adv accuracy:   62.1%
epsilon = 0.20  ->  adv accuracy:   46.5%
epsilon = 0.25  ->  adv accuracy:   31.2%
epsilon = 0.30  ->  adv accuracy:   19.1%
```



FGSM attack: accuracy vs. perturbation budget.

The visualization above shows the main trend of FGSM: as ε increases, adversarial accuracy drops sharply. This means that larger perturbation budgets allow the attacker to move each pixel farther in the direction that increases the loss, making misclassification more likely.

Notice that by $\varepsilon = 0.30$, the model is already performing very poorly compared with the clean baseline. This is the key lesson of FGSM: even a simple one-step gradient-sign attack can cause a well-trained classifier to fail badly.

Exercise 3: Reading the Accuracy Curve

Fill in your results and answer the questions.

Epsilon	Your accuracy
0.00	_____ %
0.05	_____ %
0.10	_____ %
0.15	_____ %
0.20	_____ %
0.25	_____ %
0.30	_____ %

1. At $\varepsilon = 0$, FGSM does nothing. Your accuracy should match the clean baseline from Exercise 1. Does it? If it is slightly different, why might that be?
2. Describe the shape of the curve. Does accuracy drop uniformly, or is there a range of ε

where it falls most steeply?

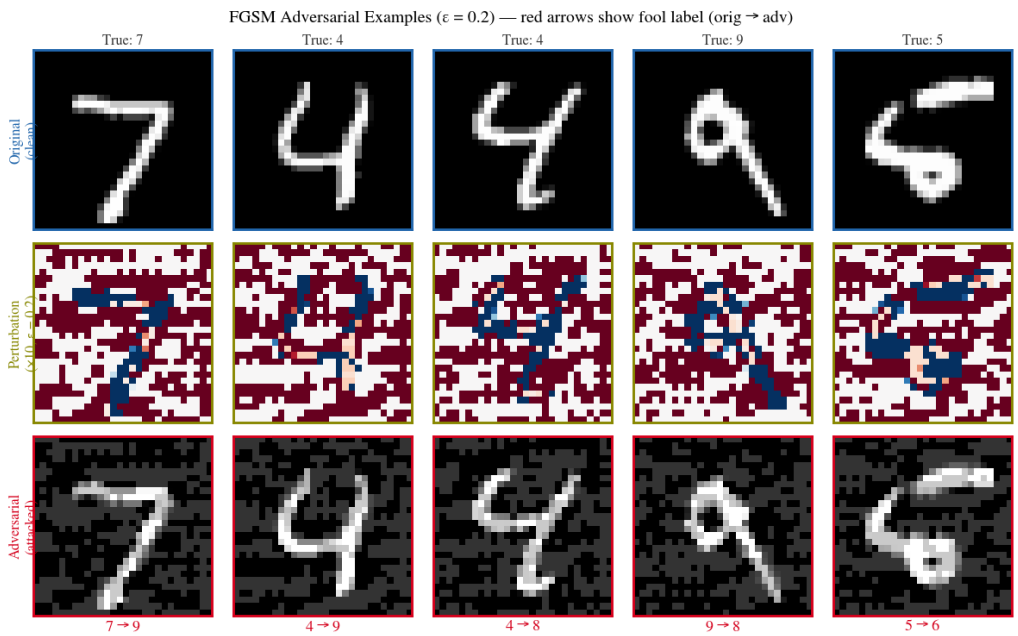
3. At $\varepsilon = 0.05$, accuracy drops but the model still works. At $\varepsilon = 0.25$, the model is barely better than random guessing. The pixel change is small in both cases. What changed?
4. **Connect to the hospital story**: The attacker shifted pixels by "amounts smaller than the noise grain of the scan." In the MNIST setting, what epsilon range do you think corresponds to "imperceptible"? Did FGSM fool the model at those epsilon values?

4.4 Visualizing the attack: what the perturbation looks like

Listing 8: Cell 7: Visualize FGSM adversarial examples in a 3-row gallery

```
# Visualization 4: Adversarial examples gallery
#   Layout: 3 rows x N_cols
#     Row 0 - Original image
#     Row 1 - Perturbation (epsilon * sign(grad)) x10 for visibility
#     Row 2 - Adversarial image
```

Example Output



FGSM adversarial examples at $\varepsilon = 0.20$.

How to read this visualization

Each column shows one successful FGSM attack example. The top row contains the original clean image. The middle row shows the perturbation pattern, amplified by a factor of 10 so that it becomes visible to the human eye. The bottom row shows the final adversarial image after the perturbation has been added.

The title above each column gives the true label of the image. The red label below each adversarial image shows how the prediction changes from the original class to the fooled class (for example, $7 \rightarrow 9$). This layout helps us compare the clean image, the perturbation, and the attacked image for the same example at a glance.

An important observation is that the perturbation is **not random static**. Even after amplification, it often follows meaningful parts of the digit's shape. This reflects the fact that FGSM is guided by the gradient of the loss: it perturbs the pixels that matter most for the model's decision.

Exercise 4: Understanding the Visualization

1. Compare the adversarial images in the bottom row with the original images in the top row. Can you visually distinguish them easily? What does this tell you about the stealthiness of the attack?
2. Look at the perturbation row in the middle. Does it look like random static, or does it have structure? Describe what you see.
3. The perturbation has to be amplified 10 times to be visible. This means in the actual adversarial image, the change is much smaller than what you see in the middle row. Given that, why is the model so badly fooled?
4. **Think about the hospital:** If an attacker added imperceptible pixel shifts to a real chest X-ray using FGSM, would a radiologist notice? Would the hospital's AI quality-check pipeline—which only measures accuracy on clean test images—detect the attack? Why or why not?

5 Why Does FGSM Work So Well?

One step. 784 pixels. Each moved by a tiny amount. Why is this so effective?

The key is dimensionality. An MNIST image lives in a 784-dimensional space. In high-dimensional spaces, decision boundaries—the invisible surfaces that separate one class region from another—are close to most points. Think of it this way: in a 2D plane, you might have to walk far to reach the edge of a region. In 784 dimensions, there are so many directions to walk that some of them reach a boundary almost immediately.

FGSM uses the gradient to find one such direction efficiently: the direction that increases the loss the fastest. One step of $\varepsilon = 0.1$ in this direction is enough to cross the boundary in most cases.

What is happening? Why the model fails

The model is not "dumb." It is doing exactly what it was trained to do—classify images based on their pixel patterns. The attack exploits the fact that the model's decision boundary in pixel space is:

1. **Close**: in 784 dimensions, it is near most images.
2. **Linear** (locally): the gradient direction—which is what FGSM uses—is a good approximation of the fastest path to the boundary.
3. **Invisible**: moving along the gradient direction looks like noise to a human but looks

like a completely different image to the model.

FGSM is fast because it takes one step. It is effective because that one step exploits all 784 dimensions simultaneously.

6 Attack 2: PGD — Many Steps, Stronger Attack

6.1 The limitation FGSM ignores

FGSM takes one big step of size ε in the gradient direction. But the loss landscape is curved. After that one step, the gradient direction has changed—the step might have overshot, landing somewhere less damaging than the true worst case.

Projected Gradient Descent (PGD, Madry et al. 2018) fixes this by taking many *small* steps, each time re-computing the gradient from the current location. After each step, it *projects* back into the ε-ball—clipping any pixel that has drifted more than ε from the original. It also starts from a random point inside the ε-ball to avoid getting stuck in a poor local solution.

An analogy: hiking to the top of a hill

FGSM is like looking at a map, pointing in the steepest uphill direction, and walking 100 meters in one go. You might end up somewhere reasonable, or you might have crossed a ridge and started going down.
PGD is like taking 40 careful steps of 2.5 meters each, checking the slope after every step, and adjusting direction. You end up much closer to the true summit.
The "summit" here is the worst-case perturbation within the ε-ball—the point that most effectively fools the model.

One iteration of PGD:

$$\mathbf{x}^{(t+1)} = \Pi_{B_\varepsilon(\mathbf{x})}\left(\mathbf{x}^{(t)} + \alpha \cdot \text{sign}(\nabla_{\mathbf{x}^{(t)}}\mathcal{L})\right)$$

α step size per iteration (typically $\varepsilon/4$)
Π_{B_ε} projection: clip pixels back so none moves more than ε from original

Typical settings: $\alpha = \varepsilon/4$, start from random point in $B_\varepsilon(\mathbf{x})$, run for $T = 40$ steps.

Predict before you run: FGSM vs. PGD

Before running the code, commit to a prediction.
At $\varepsilon = 0.1$, FGSM reduces accuracy to about 52% (from your earlier results). PGD uses the same epsilon but takes 40 steps of $\alpha = 0.025$ each.

1. Do you expect PGD accuracy to be higher or lower than FGSM accuracy at the same ε? Why?

2. After 40 steps of size 0.025, has the total budget $\varepsilon = 0.1$ been "used up"? (Hint: the projection step enforces the budget after each step, not in total.)
3. What would PGD reduce to if you set $T = 1$ and $\alpha = \varepsilon$?

My prediction for PGD accuracy at $\varepsilon = 0.1$: _____ % Reasoning: _____

Listing 9: Cell 8: PGD attack function

```
def pgd_attack(model, images, labels, epsilon=0.1, alpha=0.025, iters=40):
    """
    Projected Gradient Descent attack.
    Args:
        epsilon : total L-inf perturbation budget
        alpha   : step size per iteration (typically epsilon/4)
        iters   : number of gradient steps (typically 40)
    """
    model.eval()

    # Start from a random point inside the epsilon-ball
    # (random init avoids getting stuck in poor local optima)
    noise     = torch.zeros_like(images).uniform_(-epsilon, epsilon)
    perturbed = torch.clamp(images + noise, 0, 1).detach()
```

Listing 10: Cell 9: Compare clean, FGSM, and PGD on the same batch

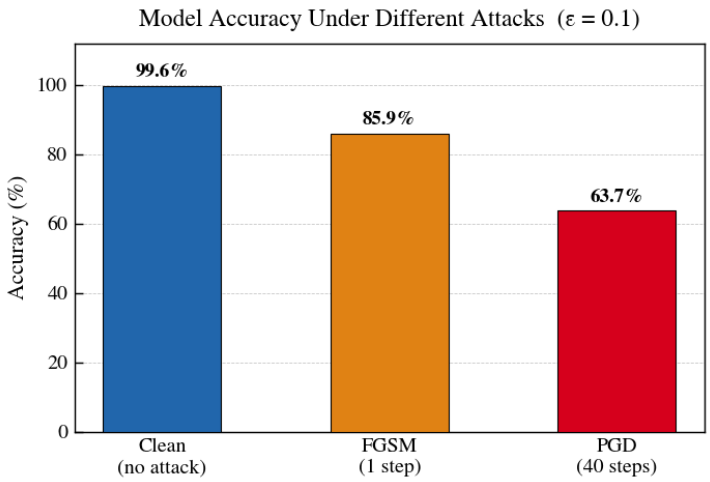
```
imgs, lbls = next(iter(test_loader))
imgs, lbls = imgs[:256].to(device), lbls[:256].to(device)

model.eval()
with torch.no_grad():
    clean_preds = model(imgs).argmax(1)
    clean_acc   = (clean_preds == lbls).float().mean().item()
    ...
)

pgd_adv = pgd_attack(model, imgs, lbls, epsilon=0.1)
with torch.no_grad():
    pgd_preds = model(pgd_adv).argmax(1)
    pgd_acc   = (pgd_preds == lbls).float().mean().item()
```

What the output should look like

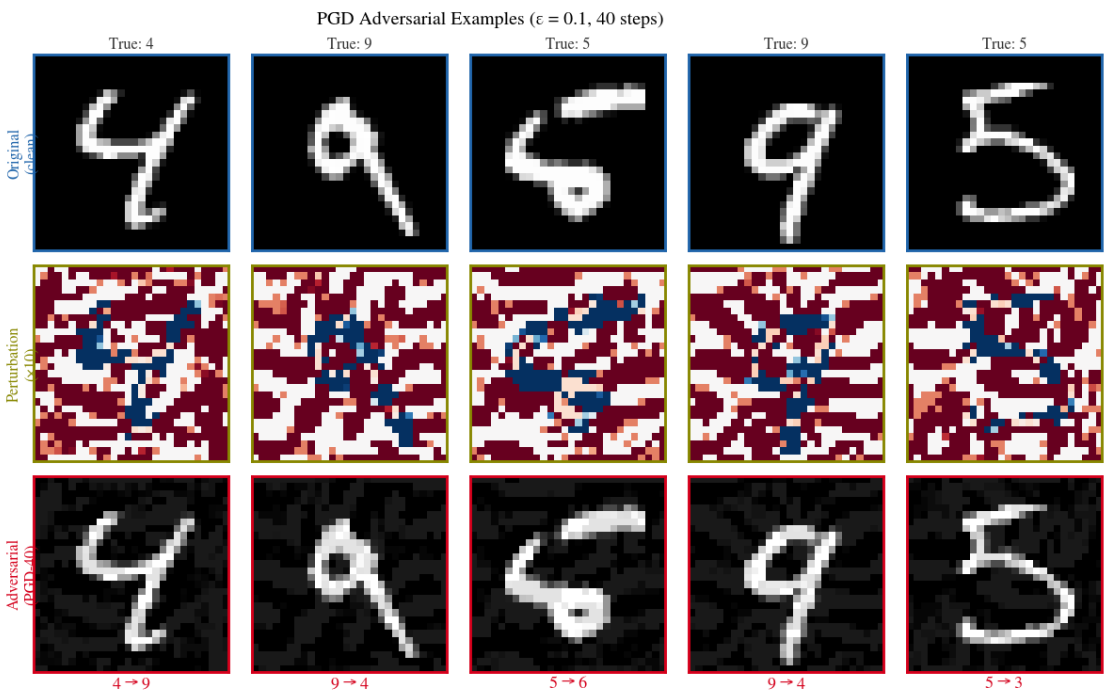
With the settings above, PGD should reduce model accuracy more than FGSM at the same perturbation budget. The example output below combines the text summary with several visual checks: a simple accuracy comparison, a step-by-step illustration of how PGD evolves over multiple iterations, and a gallery of final PGD adversarial examples.

Example Output

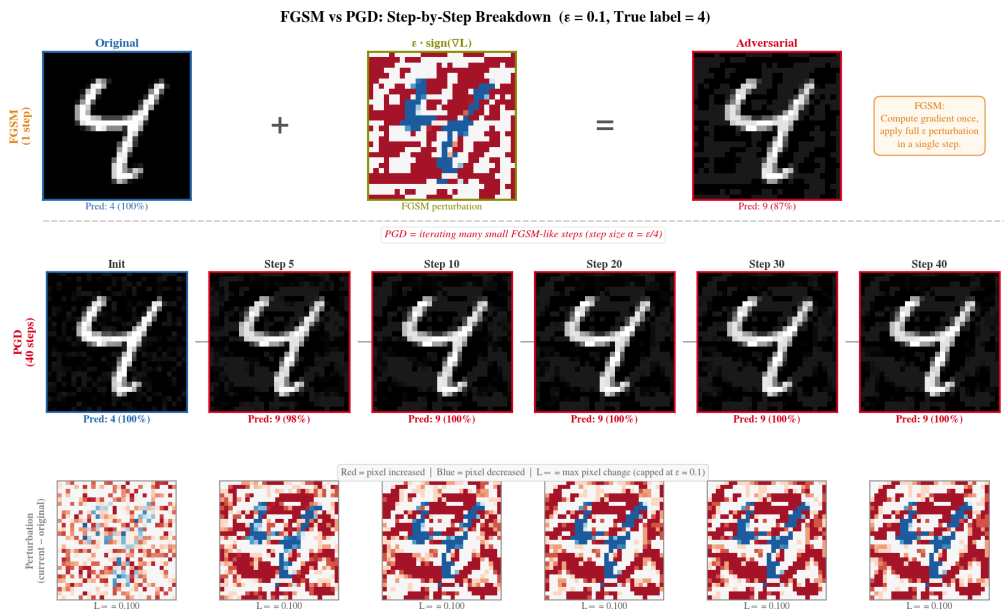
```
No attack (clean)   --   99.6%
FGSM (1 step)      0.10  85.9%
PGD (40 steps)     0.10  63.7%
```



Accuracy comparison at $\varepsilon = 0.1$.



FGSM vs. PGD step-by-step breakdown.



PGD adversarial examples after 40 steps.

Animated PGD process: View animation here

The first visualization shows the main numerical result: under the same $\varepsilon = 0.1$ budget, PGD reduces accuracy more than FGSM. This is the clearest evidence that multiple iterative updates usually find a stronger attack than a single FGSM step.

The second visualization explains *why*. FGSM computes the gradient once and applies the full perturbation immediately, whereas PGD keeps recomputing the gradient and refining the adversarial image over many small steps. This makes PGD better at searching for a more damaging point inside the same ε-ball.

The third visualization shows what the final PGD attacks look like on several examples. As in the FGSM section, each column can be read from top to bottom: original image, amplified perturbation, and final adversarial image. The prediction changes at the bottom show that the attack succeeds even though the true adversarial image still looks very similar to the clean input.

The animation link is included as an optional visual aid. It can help us see the iterative nature of PGD more intuitively than a static figure, because it shows how the image changes gradually across optimization steps.

Exercise 5: FGSM vs. PGD

Fill in the comparison table and answer the questions.

Condition	Epsilon	Steps	Accuracy
No attack (clean)	—	—	_____ %
FGSM	0.10	1	_____ %
PGD	0.10	40	_____ %

1. How does your result compare to your prediction? Based on both the bar chart and the step-by-step visualization, was PGD stronger or weaker than you expected?
2. Both attacks use exactly the same $\varepsilon = 0.1$. If you displayed both adversarial images side by side, could a human tell them apart? Does that mean the attack quality is the same?
3. **Explore:** Change the step count in `pgd_attack` from 40 to 5, then to 1. What happens to accuracy? At what step count does PGD effectively become FGSM?
4. PGD is considered the "gold standard" for evaluating robustness. Why would researchers insist on using a stronger, slower attack when evaluating a defense?

7 Defense 1: Adversarial Training

7.1 The simplest idea: show the model what it will face

If the model fails on adversarial examples because it has never seen them during training, the most direct fix is to *train on them*. At each training step, we generate PGD adversarial examples from the current batch and train the model to be correct on those adversarial inputs—not the clean ones.

This is called **adversarial training**. It essentially expands the training distribution: instead of learning from clean images only, the model learns from the worst-case perturbed versions of those images within the ε-ball.

Think about the tradeoff before running

The model has a fixed capacity—a fixed number of parameters. Previously, all of that capacity was used to classify clean images well. Now we are asking it to also classify adversarially perturbed images well.

1. Do you expect adversarial training to improve robustness? Hurt clean accuracy? Both?
2. Adversarial training is about 3–4 times slower than standard training because PGD must run for every batch. Given that cost, when would you decide it is worth it?
3. We train on PGD examples with $\varepsilon = 0.1$. What do you predict would happen if we then attacked the robust model with $\varepsilon = 0.3$?

My predictions:

Clean accuracy after adversarial training: _____ % PGD-40 accuracy: _____ %

Listing 11: Cell 10: Adversarially train a robust model (5 epochs, 7 PGD steps per batch)

```
# Expect this to take about 10-15 minutes on CPU.
# On a GPU, about 2-3 minutes.
```

```python

robust_model = SimpleCNN().to(device)
robust_opt   = torch.optim.Adam(robust_model.parameters(), lr=0.001)

for epoch in range(5):
    robust_model.train()
    running_loss = 0.0
    for imgs, lbls in train_loader:
        imgs, lbls = imgs.to(device), lbls.to(device)

        # Generate adversarial examples on the fly using PGD
        # (7 steps is a fast but effective training-time approximation)
        adv = pgd_attack(robust_model, imgs, lbls,
                         epsilon=0.1, alpha=0.025, iters=7)

        # Train on the adversarial examples, not the clean ones
        robust_opt.zero_grad()
        ...
```

Listing 12: Cell 11: Compare standard vs. robust model on clean and adversarial inputs

```python
def evaluate_both(m, imgs, lbls, pgd_eps=0.1):
    """Returns (clean_acc, pgd_acc) for a model on the given batch."""
    m.eval()
    with torch.no_grad():
        clean = (m(imgs).argmax(1) == lbls).float().mean().item()
    pgd_adv = pgd_attack(m, imgs, lbls, epsilon=pgd_eps, iters=40)
    with torch.no_grad():
        pgd   = (m(pgd_adv).argmax(1) == lbls).float().mean().item()
    return clean, pgd

...
```

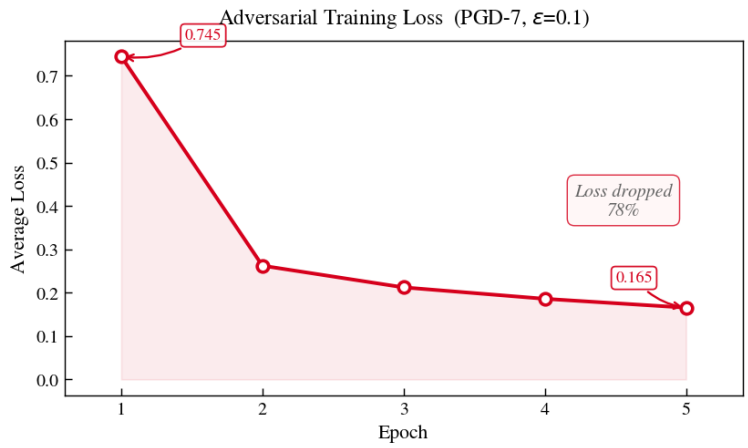
What the output should look like

With the settings above, adversarial training should make the model much more resistant to PGD attacks. The example output below combines representative training logs, the clean-versus-robust comparison, and two visual summaries: one figure showing the adversarial-training loss across epochs and one figure comparing clean and PGD-40 accuracy for the standard and robust models.

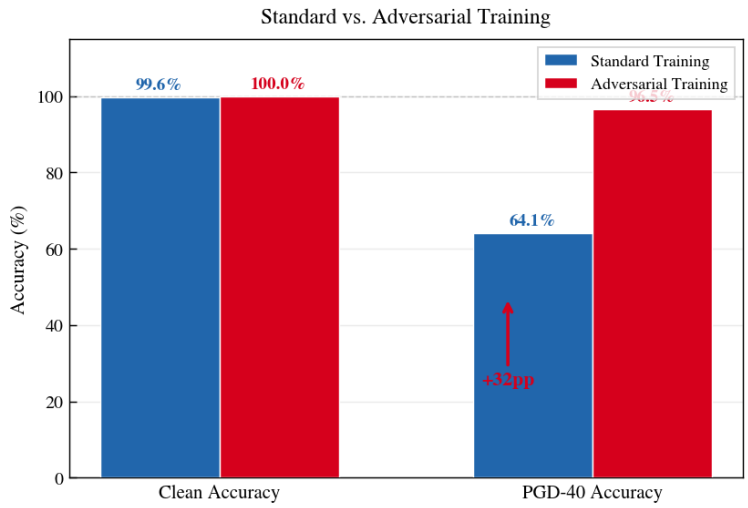
Example Output

```
Epoch 1/5  |  avg loss:   0.7446
Epoch 2/5  |  avg loss:   0.2621
Epoch 3/5  |  avg loss:   0.2121
Epoch 4/5  |  avg loss:   0.1858
Epoch 5/5  |  avg loss:   0.1654
 Adversarial training complete.
```

```
Standard training     |  clean:   99.6%,  PGD-40:   64.1%
Adversarial training  |  clean:  100.0%,  PGD-40:   96.5%
```



Standard vs. adversarial training.



Adversarial training loss across epochs.

The first figure shows the main robustness result: after adversarial training, the model performs far better under PGD-40 attack than the standard model. In this run, the robust model reaches 96.5% PGD-40 accuracy, compared with only 64.1% for the standard model. This is strong evidence that training on adversarial examples can substantially improve robustness.

The second figure helps interpret the training process itself. The adversarial-training loss starts high and then drops steadily over the 5 epochs, which suggests that the model is gradually learning to classify adversarially perturbed inputs more reliably. In this particular run, the clean accuracy of the adversarially trained model does not decrease, likely because MNIST is a relatively simple dataset and the CNN still has enough capacity to fit both the clean patterns and the adversarially

perturbed ones at this attack strength. In other words, under this small-scale setting, the model is not yet forced to sacrifice clean performance in order to gain robustness. However, in general, adversarial training may still involve a tradeoff between clean accuracy and robustness depending on the model, dataset, and attack strength.

Exercise 6: The Robustness Tradeoff

1. Fill in the table:

Model	Clean accuracy	PGD-40 accuracy
Standard training	_____ %	_____ %
Adversarial training	_____ %	_____ %

2. Did your results match your predictions? What surprised you?
3. Compare the clean accuracy and PGD-40 accuracy of the two models. In this run, did adversarial training improve robustness without hurting clean accuracy, or did you observe a tradeoff? More generally, why might that tradeoff depend on the model, dataset, and attack strength?
4. We trained with $\varepsilon = 0.1$ and tested with $\varepsilon = 0.1$. Now run `evaluate_both(robust_model, imgs, lbls, pgd_eps=0.3)`. What happens? What does this tell you about the limits of adversarial training?

8 Defense 2: Feature Squeezing

8.1 A completely different philosophy

Adversarial training changes the *model*: it learns to resist attacks. Feature squeezing changes the *input*: it strips out the attack before the model ever sees it.

The intuition: adversarial perturbations are *precise*. They are calculated to exploit a specific direction in pixel space. If you destroy that precision—by quantizing pixel values or blurring the image—the attack may lose its effect.

Two methods:

- **Bit-depth reduction**: quantize each pixel to fewer discrete levels. At 4-bit, there are $2^4 = 16$ levels instead of 256. Small perturbations may be partially or fully removed by this quantization process, depending on where the original pixel values lie relative to the quantization thresholds.
- **Spatial smoothing**: replace each pixel with the average of its $k \times k$ neighborhood. This blurs high-frequency noise—including adversarial perturbations that alternate sign between adjacent pixels.

Why is this an "empirical" defense?

Feature squeezing has no formal guarantee. There is no mathematical proof that it works against all possible attacks within a given ε. It works against specific attacks in practice, but

an attacker who knows you are applying feature squeezing can craft a perturbation that survives it.
This is called an **empirical defense**: it works experimentally against known attacks, but provides no certified protection. Compare this to adversarial training, which at least provides protection against the specific attack type used during training.

Before you run: design your expectation

1. A PGD perturbation with $\varepsilon = 0.1$ means each pixel shifts by up to 0.1. At 4-bit quantization, each level is $1/15 \approx 0.067$ wide. Will a shift of 0.1 survive quantization, or will it be rounded away?
2. If the perturbation has opposite signs on adjacent pixels (positive on one, negative on the next), what happens when you average a 3×3 neighborhood?
3. If you apply aggressive squeezing (2-bit, 5×5) to the *clean* image before the model, do you expect clean accuracy to drop? By how much?

Listing 13: Cell 12: Feature squeezing implementations and evaluation

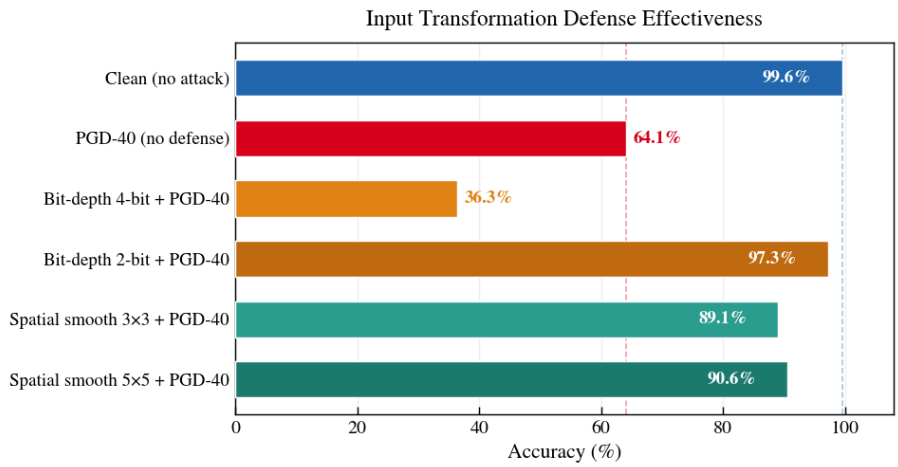
```
def bit_depth_reduction(x, bits=4):
    """
    Quantize pixel values to 2^bits discrete levels.
    bits=4  -> 16 levels (less aggressive)
    bits=2  -> 4 levels  (more aggressive)
    Any perturbation smaller than 1/(2^bits - 1) gets rounded away.
    """
    levels = 2 ** bits - 1
    return torch.round(x * levels) / levels

def spatial_smoothing(x, k=3):
    """
    Replace each pixel with the average of its k x k neighborhood.
    k=3 -> 3x3 average (mild blur)
    k=5 -> 5x5 average (stronger blur)
    """
    channels = x.shape[1]
    kernel   = torch.ones(1, 1, k, k, device=x.device) / (k * k)
    kernel_c = kernel.expand(channels, 1, -1, -1)
    return F.conv2d(x, kernel_c, padding=k//2, groups=channels)
...
```

What the output should look like

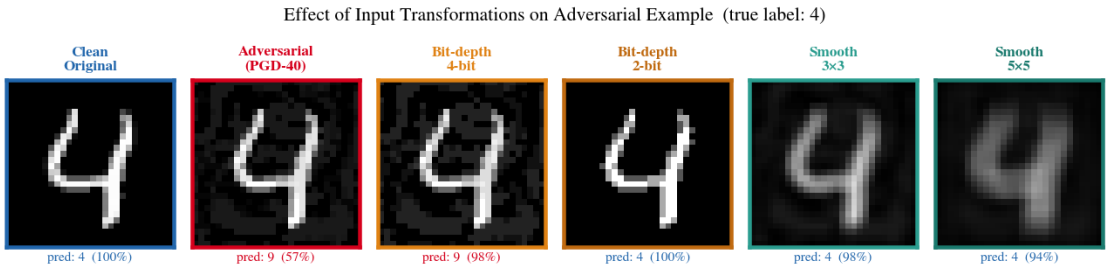
With the settings above, different input transformations should affect adversarial examples in very different ways. The example output below combines the text summary with two visual checks: an overall comparison of defense effectiveness and a single-example visualization showing how each transformation changes the attacked image.

Example Output

```
Clean (no attack)            |  99.6%
PGD-40 (no defense)          |  64.1%
Bit-depth 4-bit + PGD-40     |  36.3%
Bit-depth 2-bit + PGD-40     |  97.3%
Spatial smooth 3x3 + PGD-40  |  89.1%
Spatial smooth 5x5 + PGD-40  |  90.6%
```



Defense effectiveness under PGD-40 attack.



Effect of each input transformation on one attacked example.

The first visualization summarizes the main result of this section: some input transformations recover much more accuracy than others. In this particular run, 2-bit quantization performs surprisingly well, while 4-bit quantization performs much worse. Both smoothing-based defenses also recover a large amount of accuracy, although they remain slightly below the clean baseline.

The second visualization helps interpret what these transformations are actually doing to the image. The clean original is correctly classified, the PGD adversarial example is misclassified, and each transformed version shows how quantization or smoothing alters the attacked input. In this example, aggressive 2-bit quantization and both smoothing operations restore the correct prediction, while 4-bit quantization does not. This illustrates an important point: empirical defenses

may work well on some attacks and examples, but their behavior can vary significantly depending on the transformation strength.

Exercise 7: Understanding Feature Squeezing

1. Fill in the complete results table:

Condition	Accuracy
Clean (no attack)	_____ %
PGD-40 (no defense)	_____ %
Bit-depth 4-bit + PGD-40	_____ %
Bit-depth 2-bit + PGD-40	_____ %
Spatial smooth 3×3 + PGD-40	_____ %
Spatial smooth 5×5 + PGD-40	_____ %

2. Now apply each squeezer to the *clean* images (no attack): `acc_on(bit_depth_reduction(imgs, 4))`. Does accuracy drop on clean images? What does this tradeoff mean for deployment?
3. Compare 4-bit and 2-bit quantization. In this run, why might the more aggressive 2-bit transformation recover much more accuracy from the attack than 4-bit quantization? What possible tradeoff would you still want to check before declaring 2-bit the better choice in general?
4. **Important note**: Both defenses here are *empirical*—they work in practice against PGD but have no formal guarantee. Feature squeezing requires no retraining and can be added to a deployed model in minutes. Adversarial training requires full retraining and is much more computationally expensive. In the hospital scenario, which would you reach for first, and what would determine your final choice?

9 Putting It All Together

9.1 The defense comparison table

Complete the master summary with your results.

Defense	Clean acc	PGD-40 acc	Retraining?	Guarantee?
None (standard model)	_____ %	_____ %	No	None
Adversarial training	_____ %	_____ %	Yes	Empirical
Bit-depth reduction (4-bit)	_____ %	_____ %	No	Empirical
Spatial smoothing (3×3)	_____ %	_____ %	No	Empirical

9.2 A practical decision guide

Defense	When to use it
Adversarial training	Use when you have time and compute for retraining, and when robustness is the highest priority. Accept the small clean accuracy cost.
Feature squeezing	Use when the model is already deployed and retraining is not possible, or as a first-line defense while retraining is in progress. Expect weaker but faster protection.
Neither alone	In high-stakes settings, combine both: use adversarial training for the model and feature squeezing as an additional input filter. Two layers are better than one.

Exercise 8: Final Reflection

1. Return to your hypothesis from Section 1. How has your understanding changed? Write one paragraph explaining the hospital attack to a non-technical hospital administrator—someone who knows what X-rays are but has never studied machine learning.
2. Both defenses in this lab are empirical. A certified defense would come with a mathematical proof that *no* attack within a given ε can fool the model. Why might certified robustness be harder to achieve than empirical robustness?
3. The attacks in this lab are *white-box*: the attacker knows the model architecture and weights. In a real deployment, the attacker might only have access to the model's output (a black-box setting). Does that make adversarial attacks impossible? Easier? Harder?
4. Both FGSM and PGD produce adversarial examples that look identical to humans. If you deployed a medical AI, who should decide how much robustness to require? The ML engineer? The hospital administrator? A regulator?

10 Summary

Concept	What it does	Key insight
Adversarial example	Input crafted to fool a model while remaining difficult for humans to notice	Exploit gradient direction, not random noise
ε-ball	Budget: no pixel moves more than ε	Constrains invisibility
FGSM	One gradient-sign step across all pixels	Fast; effective in one shot
PGD	T small FGSM steps with projection	Finds worst-case in ε-ball
Adversarial training	Train on PGD examples; improves robustness	May trade clean accuracy for robustness; no formal guarantee
Feature squeezing	Preprocess input to destroy perturbation precision	No retraining; weaker guarantee

This table summarizes the key concepts, and Fig. 1 shows the whole story of the lab in one view. It starts from a clean image that the model classifies correctly, then shows how two gradient-based attacks can turn that same image into a wrong prediction, and finally compares two defense ideas that try to recover robustness.

Reading the figure from left to right: the clean baseline shows that the model works well on ordinary inputs; the FGSM panel shows that a single carefully chosen step can already reduce accuracy; the PGD panel shows that repeating smaller projected steps usually makes the attack even stronger; and the two defense panels show two different responses—either retrain the model on adversarial examples, or transform the input to weaken the perturbation before prediction. The overall message is simple: tiny, structured pixel changes can strongly affect a neural network, and defending against them requires either making the model itself more robust or reducing the effectiveness of the perturbation.

References

Goodfellow, I., Shlens, J., & Szegedy, C. (2015). Explaining and harnessing adversarial examples. *ICLR 2015*. arXiv:1412.6572

Madry, A., et al. (2018). Towards deep learning models resistant to adversarial attacks. *ICLR 2018*. arXiv:1706.06083

Xu, W., Evans, D., & Qi, Y. (2018). Feature squeezing. *NDSS 2018*. arXiv:1704.01155

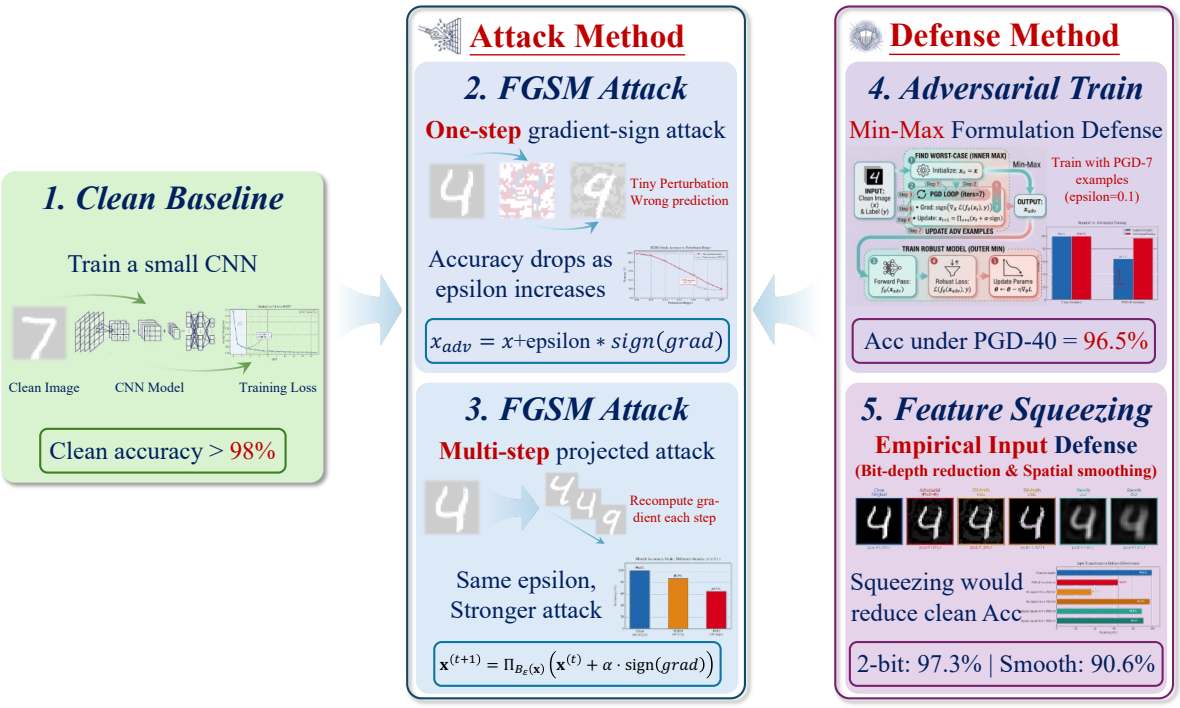


Figure 1: **Summary.**